CodeMind AI - Project Context & Development Blueprint

1. Project Overview

What is CodeMind AI?

CodeMind AI is an AI-powered Repository Intelligence Platform.

The goal is not simply to build another GitHub chat application.

The goal is to create a system capable of understanding any software repository and answering questions about it with deep repository awareness.

Examples:

- What technologies are used in this repository?
- Explain the architecture.
- Explain the workflow.
- What happens when a user logs in?
- What modules depend on this file?
- Which components will be impacted if this service changes?
- Explain this function.
- Generate architecture diagrams.
- Identify entry points.
- Explain database interactions.
- Suggest refactoring opportunities.

Ultimately, CodeMind AI should become a Repository Understanding Engine rather than a simple RAG chatbot.

---

2. Long-Term Vision

Most repository chat applications work like this:

Repository ↓ Chunk Files ↓ Vector Database ↓ LLM

This approach has several problems:

- Poor architectural understanding
- Weak dependency awareness

- Hallucinations
- No workflow understanding
- No impact analysis

CodeMind AI follows a different approach:

Repository ↓ Scanner ↓ AST Analysis ↓ Dependency Graph ↓ Dependency Analytics ↓ Tech Stack Detection ↓ Architecture Engine ↓ Workflow Engine ↓ Impact Analysis ↓ Knowledge Graph ↓ AI Assistant

The AI layer is intentionally placed near the end.

The system should understand the repository before asking an LLM to explain it.

---

3. Development Philosophy

Several important design decisions were made.

Decision 1: Build Intelligence Before AI

We are not building:

Repository → Vector DB → Chat

We are building:

Repository → Repository Intelligence → AI

The AI should consume structured repository knowledge rather than raw source code whenever possible.

Decision 2: Every Feature Must Be Testable

Every feature is exposed as an API endpoint.

This allows:

- Independent testing
- Better debugging
- Easier development
- Faster iteration

Decision 3: Build in Layers

Each completed feature becomes a dependency for future features.

Repository Import ↓ Scanner ↓ AST ↓ Dependency Graph ↓ Dependency Analytics ↓ Architecture ↓ Workflow ↓ Impact Analysis ↓ AI Assistant

---

4. Completed Features

4.1 Repository Import

Purpose

Allow users to import GitHub repositories.

Endpoint

POST /repo/import

Input

{ "github_url": "https://github.com/tiangolo/fastapi" }

Output

{ "status": "success", "path": "repositories/fastapi" }

Why It Exists

Every analysis feature requires a local repository copy.

Repository Import is the foundation of the entire platform.

4.2 Repository Scanner

Purpose

Understand repository size and composition.

Endpoint

GET /scan/{repo_name}

What It Does

Recursively scans repository files.

Calculates:

- Total files
- Total lines
- File extensions
- Language distribution

Example:

{ "total_files": 3006, "total_lines": 848402, "languages": { ".py": 1120, ".md": 1551 } }

Why It Exists

Provides:

- Repository metadata
- Language detection foundation
- Future tech stack detection signals

4.3 AST Analysis Engine

Purpose

Extract code structure from source files.

Current Scope

Python repositories.

Extracted Data

Functions

{ "name": "get_user", "line": 25 }

Classes

{ "name": "UserService", "line": 10 }

Imports

{ "module": "fastapi" }

Why It Exists

AST gives semantic understanding.

Instead of treating code as text:

Code ↓ Syntax Tree ↓ Structured Information

Future features rely heavily on AST analysis.

4.4 Import Resolution Improvement

Problem

Original implementation only stored:

from fastapi import routing

as:

{ "module": "fastapi" }

This lost important information.

Fix

Parser now stores:

{ "module": "fastapi", "full_import": "fastapi.routing" }

Impact

Dependency Graph accuracy improved significantly.

Future architecture generation becomes much more accurate.

4.5 Package Registry Service

Purpose

Differentiate internal repository imports from external dependencies.

Example

Repository contains:

fastapi/routing.py

Registry stores:

fastapi fastapi.routing

Why

Without a package registry:

from fastapi.routing import APIRouter

would be indistinguishable from third-party imports.

4.6 Dependency Graph Engine

Purpose

Build repository dependency relationships.

Technology

NetworkX Directed Graph

Example

applications.py ↓ fastapi.routing

applications.py ↓ typing

Output

{ "source": "...", "target": "...", "type": "internal" }

Why It Exists

Dependency Graph becomes the backbone for:

- Architecture Detection
- Workflow Detection
- Impact Analysis
- AI Answers

4.7 Internal vs External Dependency Classification

Problem

Not all imports are equal.

Example:

Internal:

fastapi.routing

External:

pydantic

Solution

Package Registry comparison.

Output:

{ "type": "internal" }

or

{ "type": "external" }

Impact

Allows architecture and tech stack analysis.

4.8 External Import Normalization

Problem

Imports appeared as:

typing.Any typing.Annotated pydantic.BaseModel

These are symbols, not technologies.

Solution

Normalize:

typing.Any ↓ typing

pydantic.BaseModel ↓ pydantic

Impact

Much cleaner analytics.

4.9 Dependency Analytics Engine

Purpose

Identify the most important modules.

Endpoint

GET /dependencies/{repo_name}/summary

Example

{ "module": "fastapi", "imports": 574 }

What It Means

574 modules depend on fastapi.

Why It Exists

Dependency Graph answers:

"What depends on what?"

Dependency Analytics answers:

"What is important?"

This is critical for architecture generation.

---

5. Current Architecture

Current system:

Repository Import ↓ Scanner ↓ AST Parser ↓ Package Registry ↓ Dependency Graph ↓ Dependency Analytics

All completed and functional.

---

6. Current Work In Progress

Architecture Engine

Purpose:

Convert dependency information into human-readable architecture.

Example Output:

{ "tech_stack": [...], "entry_points": [...], "core_modules": [...], "layers": [...], "architecture_pattern": "...", "summary": "..." }

Architecture Engine is the first feature that explains a repository rather than simply analyzing it.

---

7. Future Tech Stack Service Design

The initial implementation used a hardcoded mapping.

This was rejected because it cannot scale.

Example:

tech_mapping = { "fastapi": "FastAPI" }

This approach does not work for arbitrary repositories.

New Design

Universal TechStackService

Repository ↓ Dependency Files ↓ Import Analysis ↓ Technology Classification ↓ Structured Output

Output:

{ "languages": [], "frameworks": [], "databases": [], "ai_stack": [], "testing_tools": [], "build_tools": [], "containerization": [], "cloud": [] }

Supported ecosystems:

- Python
- Node.js
- Java
- Go
- Rust
- .NET
- PHP

Future-proof architecture.

---

8. Remaining Roadmap

Phase 1

Architecture Engine

Uses:

- Scanner
- Dependency Analytics

Produces:

- Tech Stack
- Entry Points
- Core Modules
- Layers
- Architecture Pattern

Phase 2

Workflow Engine

Purpose:

Explain application execution flows.

Example:

User Login Flow

User ↓ API ↓ Service ↓ Repository ↓ Database

Uses:

- AST
- Dependency Graph
- Architecture Engine

Phase 3

Impact Analysis Engine

Purpose:

Determine what breaks if something changes.

Example:

What happens if auth_service.py changes?

Output:

Dependent Modules Affected APIs Affected Services

Uses:

- Dependency Graph
- Workflow Engine

Phase 4

Knowledge Graph

Purpose:

Create repository-wide semantic relationships.

Nodes:

- Files
- Classes
- Functions
- Modules

Edges:

- Imports
- Calls
- Inheritance

This becomes the repository memory layer.

Phase 5

AI Assistant

Only after repository intelligence exists.

The assistant will consume:

- Architecture
- Workflows
- Dependency Graph
- Impact Analysis
- Knowledge Graph

instead of relying solely on vector search.

This dramatically improves answer quality.

---

9. Final Target

CodeMind AI should eventually be capable of:

- Repository Import
- Tech Stack Detection
- Architecture Generation
- Workflow Analysis
- Impact Analysis
- Dependency Analysis
- Knowledge Graph Creation
- Architecture Diagram Generation
- AI Chat
- Repository Documentation Generation

The end goal is a complete Repository Intelligence Platform that understands software repositories before attempting to explain them.